

Adaptive Resilience: Why AI Systems Must Remain Viable in Any Future

Viability as an Architectural Discipline for Adaptive AI Orchestration in High-Consequence Environments

By Jorge Enrique Flores Montano

Founder, JM Automated Solutions

~MILO™ — Modular Intelligent Learning Orchestrator (patent pending)

May 2026

ORCID iD: 0009-0003-1859-8418

DOI: 10.5281/zenodo.20117025

Public reference: <https://github.com/jmontano1/milo-architecture>

Abstract

Predictive optimization has reached structural limits as the design criterion for adaptive AI systems in high-consequence environments. Systems trained to maximize accuracy against expected future distributions fail under distribution shift, and the failure is not graceful: prediction-optimized systems collapse where the prediction was wrong. This paper argues that the next generation of adaptive AI orchestration must be designed for *viability* rather than for prediction accuracy — engineered to remain operational across futures that include the unforeseen, the rare, and the actively adversarial. The argument synthesizes three established lineages: Beer's Viable System Model [1], Hollnagel's resilience engineering [2], and Taleb's antifragility [3]. The contribution of this paper is not to restate those frameworks but to articulate viability as a concrete adaptive-AI orchestration discipline: an architecture in which audit-first command flow, modular subsystem construction with strict separation of concerns, bounded recovery pathways, tail-event preparation, and preserved operator authority together produce a system that does not require accurate prediction to remain useful. The discipline is illustrated using MILO, a patent-pending adaptive AI orchestrator [4] submitted to the U.S. Department of Energy under the Genesis Mission [5]. The unifying principle is stated plainly: *MILO does not predict the future. It remains viable in any future.*

Keywords: adaptive AI, viability, resilience engineering, antifragility, distribution shift, AI orchestration, high-consequence systems, human-in-the-loop.

Highlights.

- Articulates *viability* as a concrete adaptive-AI orchestration discipline, distinct from prediction accuracy as a design criterion for high-consequence environments.
- Synthesizes three established lineages — Beer's Viable System Model, Hollnagel's resilience engineering, and Taleb's antifragility — into a coherent design discipline operationalized through six mechanisms (M1–M6).
- Frames viability as a falsifiable design target: a system whose audit trail is incomplete, whose recovery is improvised, whose adaptation drifts unboundedly, or whose operator override is policy-level fails the principle.

- Bounds antifragility-applied-to-the-system from antifragility-applied-to-operators through eight non-negotiable operational integrity constraints.

Index Terms: adaptive AI, viability, resilience engineering, antifragility, distribution shift, AI orchestration, high-consequence systems, Beer Viable System Model, Hollnagel four cornerstones, Taleb antifragile, non-stationarity, critical infrastructure.

Plain Language Summary. Most AI systems today are designed to make accurate predictions about the future based on past data — and they fail, often catastrophically, when the future stops resembling the past. In critical-infrastructure environments (power grids, manufacturing lines, nuclear facilities, autonomous robotics, satellite operations), an AI system whose usefulness depends on accurate prediction is a brittle system. This paper argues that the next generation of AI orchestration for high-consequence environments must be designed for *viability* — the capacity to remain operational, auditable, and human-controllable under conditions the system was not trained to expect — rather than for prediction accuracy. The principle synthesizes Beer's cybernetic viability, Hollnagel's resilience engineering, and Taleb's antifragility into a concrete engineering discipline.

Relevance to U.S. National Interest. AI systems deployed in U.S. critical-infrastructure environments — the deployment context identified by the DOE Genesis Mission — face operational futures that include adversarial action, climate-driven environmental shifts, supply-chain disruption, and unforeseen sociotechnical failures. AI orchestrators whose viability is contingent on prediction accuracy are structurally unsuited for those environments. The discipline articulated here is intended to make U.S. critical-infrastructure AI viable across the futures it will actually encounter.

Status of claims. This paper synthesizes three established intellectual lineages — Beer's Viable System Model [1], Hollnagel's resilience engineering [2], and Taleb's antifragility [3] — into a *viability discipline* for adaptive AI orchestration. The lineages themselves are external references with extensive standing; the synthesis and the six mechanisms (M1–M6) articulated in §3 as a coherent design discipline whose explicit target is viability rather than prediction accuracy are the author's contribution. Mechanisms M1–M3 are foundational architectural patterns developed in companion architectural papers by the author [4]; M4–M6 are developed in greater technical depth in those companion papers, indexed at <https://github.com/jmontano1/milo-architecture>. Empirical validation of the viability discipline against actual non-stationary deployment scenarios is forthcoming work and is not claimed here. This manuscript is a preprint prior to peer review.

About MILO. MILO (Modular Intelligent Learning Orchestrator) is a patent-pending adaptive AI orchestration architecture aimed at high-consequence critical-infrastructure environments. Its central design commitment is *viability* — operational continuity under non-stationary conditions — rather than *prediction accuracy* against an expected future. The architecture is governed by eight structural principles and eight operational integrity constraints, specified in detail in the companion architectural reference and not redeveloped here. The application submitted to the U.S. Department of Energy under the Genesis Mission [Executive Order 14363, November 2025] is currently under review; status updates will be disclosed if and when public information becomes available. The mark ~MILO™ (USPTO Serial No. 99706004, intent-to-use, Class 009) and the underlying patent application are both pending. Public reference: <https://github.com/jmontano1/milo-architecture>. Author contact: jmontano@jmautomated.com.

1. Introduction

The dominant paradigm in machine-learning system design treats prediction accuracy against a held-out distribution as the central quality metric. Models are trained on past data, optimized for expected futures, deployed when accuracy crosses a threshold, and retrained when accuracy degrades. The paradigm has produced impressive systems in stationary or near-stationary domains. It has also produced a structural failure mode that is increasingly evident in operational deployments: when the deployment environment moves outside the training distribution, the system's behavior degrades without warning and often in ways that exceed the operational tolerance of the surrounding sociotechnical context.

The failure is not new. Distribution shift in machine learning has been studied for two decades [6]. What is new is the consequence profile of the systems involved. An AI orchestration system embedded in an energy grid control room, a nuclear facility, a semiconductor fabrication line, or a defense command-and-control loop is no longer evaluable solely on prediction accuracy. It is evaluable on whether it remains *viable* — whether it preserves command-and-audit continuity, maintains recoverability after disturbance, supports human authority over consequential actions, and continues to produce useful operational behavior even when the future violates the assumptions under which the system was trained.

The framing of viability as a design criterion is not original to this paper. Beer introduced the Viable System Model in 1972 [1] as a cybernetic framework for organizations that maintain identity under environmental change. Hollnagel's resilience engineering [2] formalized resilience as the capacity to *respond*, *monitor*, *learn*, and *anticipate*. Taleb's antifragility [3] formalized systems that gain rather than lose under bounded disorder. Recent work has extended antifragility into machine learning [7], proposing a dynamic-regret-based formalism for systems that improve under non-stationarity.

The contribution of the present paper is narrower and more practical. It is the application of viability, resilience, and antifragility as a concrete *adaptive-AI orchestration discipline* — a set of mechanisms an AI orchestration system must implement architecturally to remain viable in operational deployment under high-consequence conditions. The discipline is engineering-level rather than theoretical, and it is illustrated using MILO [4], the author's working implementation submitted under the U.S. Department of Energy's Genesis Mission [5].

2. Background and Related Work

The argument builds on four lineages whose contributions are summarized here, named, and then differentiated from the present contribution.

Beer's Viable System Model [1] specifies the minimum internal anatomy any organization needs to remain viable in a changing environment: a system of operations (S1), a coordination layer (S2), a control layer (S3), an intelligence/environment-scanning layer (S4), and an identity/policy layer (S5). The VSM is recursive — every viable system is composed of viable subsystems — and the framework has been applied to organizations, decentralized systems [8], and increasingly to autonomous AI architectures. The present paper is explicitly indebted to Beer. The differentiation is that the VSM is a cybernetic framework for any viable system; the discipline articulated here is the specific application of viable-system thinking to adaptive AI orchestration at the software-orchestration scale, with concrete mechanisms (modular organs, audit-first command flow, bounded reflexes) that operationalize the framework in a deployable system.

Hollnagel's resilience engineering [2] formalized resilience as four capacities: the ability to *respond* (knowing what to do when something goes wrong), the ability to *monitor* (knowing what to look for), the ability to *learn* (knowing what has happened), and the ability to *anticipate* (knowing what to expect). The Resilience Analysis Grid (RAG) operationalizes the four capacities for assessment of complex sociotechnical systems. The present paper adopts Hollnagel's four

capacities as architectural requirements for an adaptive AI orchestrator. The differentiation is at the level of architectural mechanism: each of the four capacities is mapped to a specific class of subsystem in the orchestrator (reflex response, monitoring organ, audit-driven learning, anticipatory pre-execution gating).

Taleb's antifragility [3] formalized systems that *gain* from stressors, volatility, and bounded disorder — distinct from robust systems, which merely survive. The formal substrate is the inverted Jensen inequality: convex payoff functions under noisy inputs produce positive expected returns. Applied to engineered systems, antifragility is the design property by which controlled exposure to operational variance strengthens rather than degrades the system. The present paper adopts antifragility as an architectural property of the adaptive-learning loop only — a system that learns from reviewed operational outcomes and improves its thresholds and recommendations over time. It explicitly does *not* extend antifragility into a claim that human operators inside the system should be deliberately stressed for adaptation; that application is bounded by the operational integrity constraints described in Section 5.

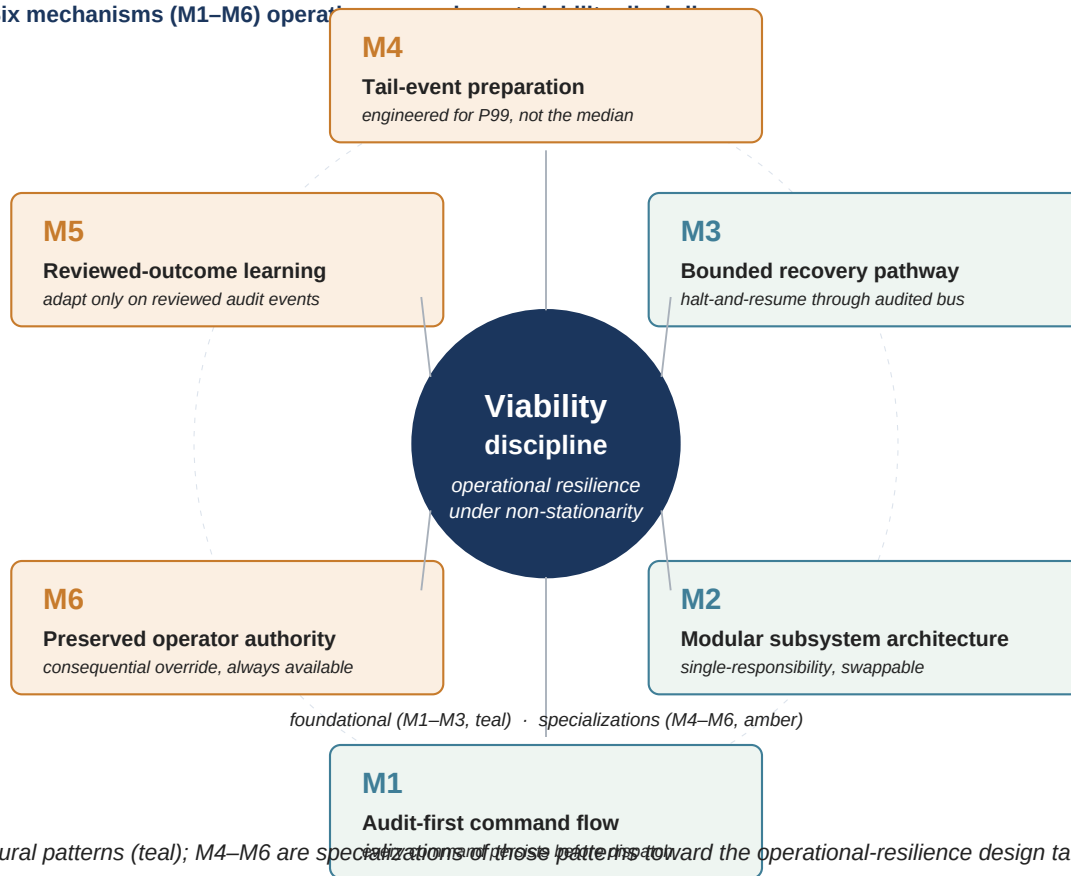
Recent ML-antifragility work [7] proposes a dynamic-regret-based formalism for machine learning systems that improve under non-stationarity. The framework operates at the level of algorithmic regret and theoretical ML guarantees. The present paper operates at the orchestration level — system-of-systems composition — and treats antifragility as an architectural property of the orchestrator's reviewed-outcome learning loop rather than as an algorithmic property of any individual model.

Standards and regulatory context anchors the discussion in operational reality. NIST SP 800-82r3 [9] specifies operational technology security requirements, prioritizing availability, integrity, and safety. NIST AI 100-1 (AI RMF 1.0) [10], particularly Appendix C on AI Risk Management and Human-AI Interaction, addresses the governance dimensions of human-AI teaming in operational environments. These standards anchor the discussion in real deployment context but are used here for design-context grounding, not as compliance claims.

3. Viability as an Orchestration Discipline

Adaptive AI orchestrators built for viability rather than prediction accuracy share six architectural mechanisms. Each mechanism corresponds to a specific failure mode of prediction-optimized systems. Several of these mechanisms — notably M6 (preserved operator authority) and M4 (tail-event preparation) — are developed in greater technical depth in companion architectural papers by the author and are indexed at the public reference <https://github.com/jmontano1/milo-architecture>; the contribution of the present paper is the unifying frame in which all six operate together as a coherent viability discipline whose explicit target is operational resilience under non-stationary conditions.

Six mechanisms (M1–M6) operating



M1–M3 are foundational architectural patterns (teal); M4–M6 are specializations of those patterns toward the operational-resilience design target (amber). The explicit target

M1 — Audit-first command flow. Every command issued by the orchestrator persists to a durable, append-only audit log before it dispatches to its target [4]. If the process terminates between dispatch and execution, replay reconstructs exact state. Prediction-optimized systems often lack this property; their decision trace is reconstructed from in-memory state and is lost when memory is lost. The architectural pattern *persist-before-deliver* — every visible state is backed by a persisted event — operates as the mechanism that makes the orchestrator's behavior auditable across futures, including the future in which the process crashed.

M2 — Modular subsystem architecture with strict separation of concerns. The orchestrator is organized into discrete, single-responsibility subsystems under a single-owner rule [4]: every source file lives in exactly one subsystem. The pattern admits component-level replacement without systemic collapse. The architectural consequence is that adaptation, repair, or addition occurs at the subsystem level; the rest of the system continues to operate. Prediction-optimized monolithic models cannot offer this — a degraded behavior often requires full retraining and full redeployment.

M3 — Bounded recovery pathways. Every adaptive subsystem admits an explicit halt-and-resume pathway dispatched through the same audited command bus as any other command, with end-to-end audit entries from critical signal through reflex through halt-executor [4]. The pathway implements the bounded-response property characteristic of Lyapunov-style stability: when the subsystem departs its equilibrium zone, the recovery path is architectural, not improvised. Adaptation that drifts unboundedly is not learning — it is failure.

M4 — Tail-event preparation. Empirical statistics in complex systems show that consequence profiles are heavy-tailed: the 99th-percentile event dominates the total loss [11]. Viable orchestrators implement rolling-window degradation detection, periodic self-monitoring on a bounded cadence (rather than operator-triggered checks), and bounded reporting under tail

events. In MILO, an integrity-monitoring subsystem runs a periodic self-monitoring scheduler on a bounded cadence with reflex auto-declare on findings; the architecture is engineered for the 99th-percentile event, not the median.

M5 — Reviewed-outcome learning. Adaptive learning operates on reviewed operational outcomes — outcomes for which the audit trail is intact and the human-authoritative review has been recorded. The system improves its thresholds, recommendations, and reflex parameters based on outcomes that have been audited, not on raw operational data. This is the architectural form of antifragility within the learning loop: the system *gains* from bounded operational variance under the explicit precondition that the variance is reviewed and the gain is supervised.

M6 — Preserved operator authority. Every consequential action passes through an explicit pre-execution gate that admits an operator override at any time, with the override logged for audit but never used to trigger adverse personnel or operational consequences [4]. The architectural pattern named *operator authority is the invariant* operates as the safeguard that prevents the orchestrator from degrading from useful-supplement to opaque-decision-maker as the deployment matures. Prediction-optimized systems often lack a structural override; the override becomes a policy promise rather than an architectural property.

Taken together, the six mechanisms operationalize viability as an architectural discipline rather than as a marketing claim. None of the six requires accurate prediction of the future. Each contributes to the system's capacity to remain useful in a future the system was not trained to expect.

3.1 Distinction from Prior Orchestration Work

Orchestration as an engineering category is long-established. Beer's Viable System Model dates to 1972 [1]; service orchestration in distributed systems is decades old; workflow orchestration in industrial automation, business process management, and cloud-native computing are mature fields each with their own conventions and tools. The contribution of this paper is not orchestration itself. It is the *viability discipline* applied to adaptive AI orchestration in high-consequence operational environments where the orchestrator must remain operational, auditable, and human-controllable under operational futures the orchestrator was not trained for. The discipline is specifically the six architectural mechanisms (M1 through M6) operating together in service of an explicit design target — *viability*, not prediction accuracy — under deployment contexts (advanced manufacturing, grid reliability, human-in-the-loop AI for critical infrastructure) where the consequence profile of failure is severe and the operational futures are non-stationary. The novelty of the present paper relative to prior orchestration work is the architectural application of viability discipline to this specific class of deployment, not the orchestration substrate itself.

4. The Unifying Operational Principle

The discipline above is summarized by a single operational principle, stated as the unifying claim of the framework submitted by the author under the DOE Genesis Mission [5]:

MILO does not predict the future. It remains viable in any future.

The principle requires a precise distinction. MILO is an *orchestrator* — a system that routes commands among subsystems (human operator ↔ command bus ↔ target organ), persists each command before dispatch, runs reflex predicates before fanout, and preserves operator authority over consequential actions. The orchestrator does not itself produce predictive forecasts of operational futures. The AI models invoked through the orchestrator may produce predictions, but the orchestrator's viability does not depend on those predictions being accurate; it depends on the orchestrator's architectural properties — audit completeness, bounded recovery, modular

organ structure, tail-event preparation, reviewed learning, preserved operator authority — holding under any operational future the orchestrator encounters.

The principle is not a prediction claim. It is a design target: avoid single-point collapse, preserve command-and-audit continuity, maintain recoverability after disturbance, improve from reviewed events, and preserve operator authority across the system's deployment lifetime. The principle is falsifiable: a system whose audit trail is incomplete, whose recovery is improvised, whose adaptation drifts unboundedly, or whose operator override is policy-level rather than architectural, fails the principle and is therefore not viable in the sense developed here.

The v.5 extension submitted under the Genesis Mission adds a parallel principle for the operator-cognitive performance layer of high-consequence systems: the same architectural discipline that produces system-level viability is extended to the human operators inside the system — *not* by stressing operators for adaptation (the inverse of antifragility-applied-to-people), but by supporting operators against cognitive load, fatigue, and task-state misalignment under the eight operational integrity constraints (consent, individual baseline, no surveillance, operator authority, transparency, data sovereignty, override, independent oversight) [5]. Section 5 addresses the governance dimensions of this extension explicitly.

5. Governance: Antifragility Does Not Mean Stress Dosing Operators

A framework that adopts Taleb's antifragility as an architectural property in an industrial-AI context must be explicit about the boundary between system-level antifragility and operator-level stress exposure. The boundary is non-negotiable in the framework presented here. Antifragility applies to the orchestrator's reviewed-outcome learning loop. It does *not* apply to the human operators embedded in the system. Operator-facing components of the orchestrator operate under eight non-negotiable operational integrity constraints [5]:

(1) *No coercion, ever* — the system issues recommendations, never compels. (2) *Individual baseline only* — cognitive-state-aware decision support measures against the operator's own established performance baseline, never against a population norm, government standard, or employer productivity target. (3) *No surveillance architecture* — the system is designed as a performance-support tool, not a monitoring infrastructure. (4) *Operator authority is the invariant* — the system expands effective decision options; it never narrows or preempts them. (5) *Operational transparency* — every recommendation includes a plain-language explanation of what signal was detected and what options the operator has. (6) *Data sovereignty* — operator-layer performance data belongs to the institutional program under documented data governance. (7) *Override always available* — operators can override any recommendation at any time, with overrides logged for audit but never used for adverse personnel action. (8) *Independent oversight* — operator-layer deployments require institutional ethics board review, published consent frameworks, and periodic third-party audits.

These constraints are specified as architectural commitments designed to be implemented as enforceable safeguards in deployment builds, not as policy-level promises. The architectural target is structural enforcement — code-level invariants bound to the design specification — so that violations should require rebuilding from source rather than toggling a runtime flag. The framework's claim is not that operators should be made antifragile by deliberate exposure to stress; it is that the *system* should be made antifragile by reviewed adaptation, while the operators inside the system are supported, not optimized.

6. Implications and Discussion

The discipline of viability as an adaptive AI orchestration property has implications for the design, evaluation, and deployment of AI systems in high-consequence environments. For *design*, it implies that the architectural choices made before training — the command bus, the audit substrate, the modular subsystem boundaries, the reflex layer, the operator override path — bound the system's eventual viability more tightly than any subsequent algorithmic improvement can recover. A system designed for prediction-accuracy-only cannot be retrofitted into a viability-grade orchestrator; the substrate must be present from the beginning.

For *evaluation*, it implies that prediction accuracy is necessary but insufficient. An adaptive AI orchestration system must be evaluated on its mean-time-to-recovery after disturbance, its audit-trail completeness, its tail-event preparedness, its operator-override exercise frequency under operational conditions, and its capacity to improve from reviewed outcomes — none of which are visible in a held-out accuracy metric. Operational measurement requires definitions; this paper proposes the *evaluation axes* rather than the operational metrics themselves, with each axis to be defined per deployment context. Candidate operational metrics in the framework's supporting documentation — including operator-attributable fault initiation rate, recoverable loss minutes, and decision-quality degradation against individual baseline — are specified with explicit numerator, denominator, observation window, attribution rule, and baseline comparison, and are not invoked here without those definitions.

For *deployment*, it implies that high-consequence environments — energy grid control rooms, nuclear facilities, advanced manufacturing lines, autonomous robotics under human supervision, satellite and space operations, national laboratory experimental campaigns — should not deploy adaptive AI systems that lack the six architectural mechanisms described in Section 3. The DOE Genesis Mission's emphasis on AI in advanced manufacturing, grid reliability, and human-in-the-loop systems [5] is precisely the deployment context in which viability-discipline architecture, rather than prediction-optimization, is operationally appropriate.

6.1 Limitations

This paper proposes a viability discipline; it does not present an empirical evaluation. Four specific limitations bound the present contribution: (i) the six mechanisms (M1–M6) are articulated at the architectural level; their per-mechanism operational metrics (mean-time-to-recovery after disturbance, audit-trail completeness rate, tail-event preparedness frequency, reviewed-outcome adaptation rate, override exercise rate) are specified as *evaluation axes* in §6 but are not measured against any deployed instance in this paper; (ii) the synthesis of Beer [1], Hollnagel [2], and Taleb [3] is the author's reading of those lineages and is not endorsed by those authors; alternative syntheses are plausible; (iii) the operator-cognitive performance layer (referenced via the v.5 extension in §4) is design-stage and bounded by the governance constraints in §5, not by shipped operational practice; (iv) the submission of MILO under the DOE Genesis Mission [5] is under review; no acceptance or grant outcome is claimed. Empirical validation of the viability discipline against actual non-stationary deployment scenarios is forthcoming work.

7. Conclusion

This paper has argued that adaptive AI systems in high-consequence environments must be designed for viability rather than for prediction accuracy. The argument is built on Beer's Viable System Model [1], Hollnagel's resilience engineering [2], and Taleb's antifragility [3], with adaptations to the specifics of AI orchestration: audit-first command flow, modular subsystem construction with strict separation of concerns, bounded recovery, tail-event preparation, reviewed-outcome learning, and preserved operator authority. The discipline is operationalized in MILO [4], submitted under the U.S. Department of Energy's Genesis Mission [5]. The unifying principle — *MILO does not predict the future; it remains viable in any future* — is a design target, not a prediction claim. Related architectural directions by the author — multi-source cryptographic entropy sourcing, latency-aware authentication in industrial control, supervisory primacy for human-in-the-loop AI orchestration, and the structural principles of adaptive AI architecture — each inherit the viability discipline as their substrate.

Data Availability

All architectural materials, source manuscripts, the reference implementation, and accompanying figures are openly available at <https://github.com/jmontano1/milo-architecture> and permanently archived at Zenodo (DOI: [10.5281/zenodo.20117025](https://doi.org/10.5281/zenodo.20117025)). No private datasets are referenced; the architectural framework itself is the subject of this paper. Patent rights for the underlying MILO software architecture are reserved; the ~MILO trademark is held under USPTO Serial No. 99706004 (intent-to-use, Class 009).

References

- [1] S. Beer, *Brain of the Firm*, 2nd ed. Chichester, UK: Wiley, 1981.
- [2] E. Hollnagel, D. D. Woods, and N. Leveson, Eds., *Resilience Engineering: Concepts and Precepts*. Aldershot, UK: Ashgate, 2006.
- [3] N. N. Taleb, *Antifragile: Things That Gain from Disorder*. New York, NY: Random House, 2012.
- [4] J. E. Flores Montano, *MILO (Modular Intelligent Learning Orchestrator)*, JM Automated Solutions. Patent pending. Submitted under the U.S. Department of Energy Genesis Mission, 2026.
- [5] U.S. Department of Energy, "The Genesis Mission: Transforming Science and Energy with AI," Office of the Under Secretary for Science, Executive Order 14363, November 2025. [Online]. Available: <https://www.energy.gov/genesis>
- [6] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. Cambridge, MA: MIT Press, 2009.
- [7] M. Jin, "Preparing for Black Swans: The Antifragility Imperative for Machine Learning," *arXiv preprint arXiv:2405.11397*, May 2024.
- [8] K. Nabben and M. Zargham, "Applying Stafford Beer's Viable System Model to Decentralized Organization," *BlockScience*, Apr. 2022. [Online]. Available: <https://blog.block.science/applying-stafford-beers-viable-system-model-to-decentralized-organization/>
- [9] National Institute of Standards and Technology, *Guide to Operational Technology (OT) Security*, NIST SP 800-82r3, Sep. 2023. doi:10.6028/NIST.SP.800-82r3
- [10] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, National Institute of Standards and Technology, Jan. 2023, Appendix C: AI Risk Management and Human-AI Interaction. doi:10.6028/NIST.AI.100-1

[11] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-law distributions in empirical data," *SIAM Review*, vol. 51, no. 4, pp. 661–703, 2009. doi:10.1137/070710111

Author Biography

Jorge Enrique Flores Montano is the founder of JM Automated Solutions and the inventor of MILO (Modular Intelligent Learning Orchestrator), a patent-pending adaptive AI orchestration system. He holds a Master of Science in Industrial Engineering (December 2018), with a minor in Applied Statistics and a certification in Public Utilities Regulation Economics, and a Bachelor of Science in Mechatronics Engineering (May 2016), with a minor in Industrial Automation. He is a CONACYT-SENER federal scholar (selected 90 of 2,000 applicants) and a member of Alpha Pi Mu, the U.S. industrial-engineering honor society. He has over seven years of hands-on deployment experience in industrial vision and automation systems across the food, beverage, pharmaceutical, and medical-device manufacturing domains. His work in adaptive AI architecture focuses on cryptographic entropy sourcing, industrial control authentication, supervisory primacy for human-in-the-loop AI, and the structural principles of viable adaptive AI orchestration. MILO has been submitted under the U.S. Department of Energy's Genesis Mission as a candidate architecture for AI-enabled critical-infrastructure systems.

Conflict of Interest and Funding Disclosure

The author is the inventor of MILO (patent pending) and the founder of JM Automated Solutions. The discipline proposed in this paper is a contribution from a working development program in which the author retains sole authorship and inventive interest. No external funding was received for the preparation of this manuscript. The author retains all rights to MILO and to the discipline articulated herein.

© 2026 Jorge Enrique Flores Montano · JM Automated Solutions · ~MILO™ is a trademark of Jorge Enrique Flores Montano · USPTO Serial No. 99706004 · Patent Pending.